The Soft Continuous Equal-Area Constraint (A2)

Thirty-two times faster, and twice fatally wrong: a measured rejection at N=100

August 2026

Abstract

Phase 1 spends 93.3% of its wall time in an iterative alternating projection whose only job is to enforce equal *continuous* cell mass — a constraint whose delivered meaning, an equal-area winner-take-all partition, is now guaranteed independently and exactly by the balanced readout at extraction time. This report measures what happens when that constraint is weakened to a soft quadratic penalty, leaving sum-to-one plus box: a closed-form, non-iterative per-vertex simplex projection. At N=100 the change is **32**× faster and kills no cells — the pre-registered falsifier does not fire — but it is **rejected**, on two *independent* defects that were not visible in the headline numbers. (1) The penalty's gradient is a *uniform per-cell field*, so a cell short on area is boosted everywhere at once, including at a distant foothold: fragmented cells accumulate $0 \to 4$ exactly as imbalanced cells fall $79 \to 0$. The flow buys area balance with disconnected territory. (2) The Euclidean simplex projection maps a whole small-step neighbourhood of a crisp iterate back onto the same simplex vertex, so $E_{\text{trial}} = E$ and the Armijo test can never be satisfied: *every* level collapses to a step of 9.095×10^{-13} and freezes, and the energy-plateau trigger then fires on the frozen energy and reports convergence. Most of the 32× is therefore not a speedup at all but a dead optimizer. Defect (1) was predicted in advance by the locality criterion of `docs/reference/winner_take_all_partition_gap.md` §9b, which now has two independent confirmations.

Status — measured; verdict REJECTED

All numbers below are measured at N=100 against a matched control that was already a validated deliverable. Both defects are measured, not inferred. The approach is closed: the code remains on the branch behind a default-off flag (`soft_area_constraint`) as the record, and should not be enabled. **One narrower variant remains open** at the time of writing (exact treatment at level 0, soft from level 1); §6 states what it does and does not escape.

Provenance

Date	2026-08-10 / 2026-08-11
Surface	torus $T(R{=}1.0, r{=}0.6)$, $N{=}100$, $\lambda{=}5.1$, seed 84172851, seeded init, 5 levels ($100{\times}96 \to 348{\times}328$, $V{=}114{,}144$)
Control	`results/run_20260709_081548_surftorus_npart100_..._lam5.1_seed84172851` — the validated $N{=}100$ deliverable: 0 dead / 0 weak / 0 imbalanced / 0 fragmented, worst cell 0.78%, exported Phase 2 perimeter 185.2546. Its connectivity gate postdates the run and was computed retroactively for this report.
A2 arm	config `parameters/torus_100part_coarse_seeded_softarea.yaml` — identical in every relaxation knob except `soft_area_constraint: true`, `soft_area_mu: 245.9`.
Probe	the same config with `refinement_levels: 1`, `max_iter: 10000`, `enable_refinement_triggers: false`, `h5_save_stride: 250` — level 0 run past its trigger point to separate "stopped early" from "settled".
Scripts	`docs/experiments/05-soft-area-constraint/make_figures.py` (figures); `testing/calibrate_soft_area_mu.py` (μ); `testing/check_fragmentation.py` (gates); `testing/test_soft_area_constraint.py` (correctness).
Versions	Python 3.12.13, numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, h5py 3.16.0; macOS 15.3.1 arm64 (Mac mini), `OMP_NUM_THREADS=2`.
Anchors	control Phase 1 wall 48,131.8 s (sum of `level_wall_s`); A2 Phase 1 wall 1,496.6 s; A2 level-0 energy frozen at 119.5651878371 from iteration 2,750 to 9,999; every A2 level's final step $= 9.0949 \times 10^{-13}$.

Contents

1 The question

The Phase 1 timing profile is lopsided. At N=300, λ=11.5 (`docs/math/04-phase1-timing-profile/`), the alternating projection is 93.26% of wall time — 21.96 h across 6,762 major iterations at a mean of 46.3 projection inner iterations per call — while energy, gradient and backtracking together are 6.49%. A later five-level N=300 run put the projection share at 94.80% of 57.3 h. The prize is real and it is essentially the whole of Phase 1.

That projection enforces equal *continuous* cell mass, $\sum_i v_i u_{i,k} = \bar{A}$. Its purpose is to deliver an equal-area *winner-take-all* partition. Since the balanced readout (`src/partition/balanced_readout.py`) now delivers exactly that, by construction, to one-vertex granularity, at any N, at extraction time and in seconds, the constraint's job is done twice. So: weaken it to a soft penalty and keep only sum-to-one plus box, which is a *closed-form* per-vertex simplex projection with no inner loop at all.

The pre-registered falsifier. Runt (area-imbalanced) cells were *expected* under a soft constraint and are not disqualifying — the readout repairs them. The hypothesised danger was different: the hard mass constraint may be the only thing keeping a cell *alive* early in the flow. Any dead or weak cell would fail the approach regardless of speedup. That falsifier is stated in the config header and was fixed before the run.

2 Method

2.1 The change

Equal area moves from the feasible set into the objective:

$$P_{\text{area}} = \frac{\mu}{2} \sum_k r_k^2, \qquad r_k = \frac{v^\top u_k - \bar{A}}{\bar{A}}, \qquad \frac{\partial P_{\text{area}}}{\partial u_{i,k}} = \frac{\mu}{\bar{A}^2}\, v_i\, (v^\top u_k - \bar{A}),$$

and the per-trial projection in the line search becomes the Euclidean projection of each row onto the unit simplex (sort-and-threshold), replacing the alternating projection. Two implementation choices matter for reading the results:

- The **entry projection** (once per level, on the initial iterate) stays exact in both arms, so each level starts from the identical feasible point and the comparison isolates the flow.
- `constraint_fun` **drops the area block** when the flag is on, so `FEAS` keeps describing the constraints actually enforced. Otherwise `FEAS` would sit permanently above `refine_constraint_tol` and the refinement trigger would silently mean something different in the two arms, invalidating the comparison.

2.2 Calibration of μ

μ is calibrated, not chosen: `testing/calibrate_soft_area_mu.py` picks the μ at which the penalty force reaches parity with the base energy's $\|g\|_\infty$ at a 5% relative area deviation — the discrete-area gate threshold — evaluated at the level-0 seeded initial condition. This gives $\mu = 245.9$ at N=100 (and 895.3 at N=30; μ scales with vertices-per-cell and is not N-invariant). Stiffness was checked rather than assumed: the penalty adds ≈ 302 to the Hessian's top eigenvalue, permitting steps to 6.6×10^{-3}, while this config's level-0 accepted steps already fall to 4.9×10^{-4} — so the penalty is not the step-limiting term. *This matters for §5: the line-search collapse reported there is not a stiffness artifact of μ.*

2.3 Correctness gates before any run

`testing/test_soft_area_constraint.py`: the simplex projection against the KKT conditions (stationarity residual 4.4×10^{-16}, feasibility 2.7×10^{-15}, box-exact, idempotent); the penalty gradient against central differences (3.3×10^{-8}); the wiring against an independently written closed form; and flag-off bit-identity of energy, gradient and constraint vector.

3 Result 1 — the headline, which is misleading

	control	A2
Phase 1 wall	48,132 s (13.37 h)	**1,497** s (0.42 h)
speedup	—	**32.2**×
mean projection inner iterations	34.0	1.03
dead / weak cells	0 / 0	**0/0**
imbalanced (worst)	0 (0.78%)	0 (1.47%)
continuous area drift	∼0 (hard)	2.06%
sum-to-one feasibility	1.3×10^{-10}	$\mathbf{6.7 \times 10^{-16}}$
fragmented cells	**0**	**3** (worst stray 46.3%)

The falsifier did not fire: no cell died. The soft constraint also held its own target far better than required (continuous drift 2.06%, discrete imbalance 1.47%), and sum-to-one came out six orders of magnitude *tighter* than the control, because the closed-form projection is exact where the alternating one merely reaches its stopping tolerance.

It fails on a third axis instead. Three cells are split. The balanced readout does repair them — 0 imbalanced, 0 fragmented — but at 13,540 moves against the control's 51, 213 blocked against 0, hitting the 50-sweep cap, and costing +22.0% label-boundary length against the control's +0.13%. Phase 2 inherits that geometry and **aborts**: IPOPT reports `EXIT: Restoration Failed!` at topology iteration 3 of 20, having exhausted `max_opt_iter` on the two before it. The best perimeter reached, 197.40, is where the solver died, not a converged value; the control converges to 185.2546 in 20 iterations with no solver failure.

4 Result 2 — defect (1): balance is bought with disconnection

Running level 0 past its trigger point, with triggers disabled, separates "stopped early" from "settled" and shows where the splits come from.

iteration	250	500	1500	2500	2750	→9999
fragmented cells	0	1	2	3	**4**	**4**
imbalanced cells	79	63	22	1	**0**	**0**

Fragmentation appears at iteration 500 and accumulates *monotonically as imbalance is driven out*, reaching 4 cells by iteration 2,750 and staying there — worse than the 3 the production arm ended with, so running longer makes it worse, not better. The mechanism is in the gradient: the penalty contributes $(\mu/\bar{A}^2)\, v_i\, (v^\top u_k - \bar{A})$, a field that is *uniform across cell k* up to the vertex weight v_i. A cell short on area is therefore boosted everywhere at once — including wherever it holds a small distant foothold, which grows into an island.

This was predicted. §9b of `docs/reference/winner_take_all_partition_gap.md` was written before A2 existed, to explain why the (rejected) discrete-area trim manufactured islands, and it states the test: *through what operator does the correction reach the field, and is that operator*

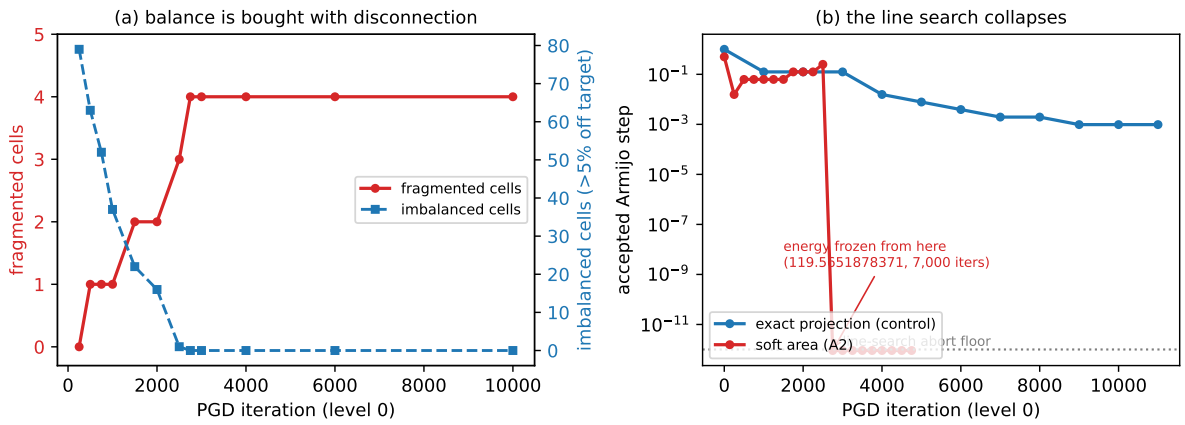

Figure 1: The two independent defects. **(a)** At level 0 the fragmented-cell count rises $0 \to 4$ exactly as the imbalanced-cell count falls $79 \to 0$: the flow buys area balance with disconnected territory, and then freezes there. **(b)** The accepted Armijo step collapses from 2.5×10^{-1} to 9.095×10^{-13} between iterations 2,500 and 2,750 and the energy freezes to the digit, while the control's step decays gracefully across 30,000 iterations and never collapses.

local? The trim reached the field through the projection, a nonlocal operator. A2 reaches it through a uniform per-cell push — nonlocal in exactly the same sense. Two mechanisms that look nothing alike, one criterion, one symptom. See §7.

5 Result 3 — defect (2): the line search collapses

The second defect is independent of the first and is the more serious one for interpreting the headline.

level	iterations run	step collapses at	final step
A2 level 0	2,575	2,548	9.095×10^{-13}
A2 level 1	282	257	9.095×10^{-13}
A2 level 2	211	182	9.095×10^{-13}
A2 level 3	184	154	9.095×10^{-13}
A2 level 4	257	232	9.095×10^{-13}
control level 0	30,000	**never**	healthy

Every A2 level dies some 25–30 iterations before its "convergence" trigger fires. In the probe, the energy is bit-identical at 119.5651878371 from iteration 2,750 through 9,999 — seven thousand iterations in which nothing is accepted — with a mean of 30.3 backtracks per iteration and 11.3% of wall time spent backtracking. The control never collapses at all.

Why. Once the field is crisp, the iterate sits at a vertex of the box-truncated simplex. The Euclidean projection maps an entire small-step neighbourhood of that point back onto the same vertex, so $x_{\text{trial}} = P(x - sg) = x$ exactly and $E_{\text{trial}} = E$, while the Armijo test demands $E_{\text{trial}} \le E - c\,s\,\|g\|^2$ with $c\,s\,\|g\|^2 > 0$. No step size can satisfy it. The search backtracks ~ 40 times to below the 10^{-12} abort floor and repeats forever. The alternating projection's multiplicative renormalisation has no such fixed point: it keeps entries strictly positive, so small steps produce small but nonzero motion.

Consequence for the headline. A2's 32× decomposes into 3.1–6.4× per-iteration cost and 4.7–11.7× fewer iterations. The second factor is *not a saving*: it is the optimizer dying at every level and the energy-plateau trigger, which cannot distinguish a frozen energy from a converged one, reporting the failure as convergence.

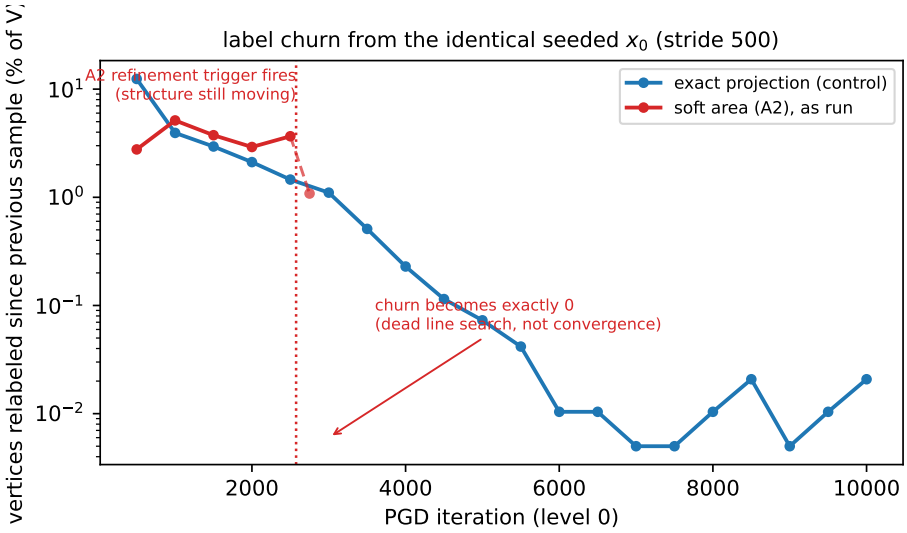

Figure 2: Winner-take-all label churn at level 0 from the identical seeded x_0. The control decays monotonically; the soft-area arm does not, and was still relabelling 3.7% of vertices per 500 iterations when its trigger fired at iteration 2,575. Past that point the churn does not become small — it becomes *exactly* zero, for 7,000 iterations, which is the signature of a dead line search rather than a settled structure.

A diagnostic worth stealing. Exactly-zero churn is qualitatively different from small churn, and it is the cheapest tell that an optimizer has stopped rather than converged. Convergence gets *cheaper* per iteration; this got 13× more expensive per iteration at the moment it stopped moving. A guard now warns when no step is accepted for 50 consecutive iterations (`src/optimization/pgd_optimizer.py`), so this failure cannot again be mistaken for a speedup.

6 What remains open

One variant is not refuted by the above: run level 0 on the *exact* treatment and levels 1–4 on the soft one (`soft_area_from_level: 1`). Fragmentation is born at level 0 (§4), and A2's finer levels never added splits beyond what level 0 handed them, so a clean level-0 structure may survive. Both halves of the treatment are gated together deliberately — penalty *and* projection — since they are separate defects and swapping only one would carry the other into the level it was built to protect.

Its prognosis is nonetheless poor, and for a reason this report already establishes: levels 1–4 keep the Euclidean projection and therefore defect (2), so they should collapse after 150–250 iterations each and do almost no refinement. The measurement is cheap (≈2 h) and is being run to convert that prediction into a number.

7 Conclusions

1. **The stated falsifier was the wrong one.** No cell died — the hard mass constraint is not what keeps cells alive. A2 failed on connectivity and on optimizer health, neither of

which was the pre-registered risk. Pre-registering a falsifier remains right; expecting it to be the thing that fires is not.

2. **Connectivity is the fragile invariant of this problem.** Nothing in the energy, the constraints, or either replacement protects it. Two independent attempts to cheapen Phase 1 have now failed by manufacturing disconnected cells: the territory-aware term via nonlocal area transport through the projection, and A2 via a uniform per-cell push. Different mechanisms, identical symptom.
3. **The locality criterion is predictive, not retrospective.** §9b stated it before A2 existed and it called A2's failure mode correctly. It should be applied as a *screening test* to any future candidate, before code: *through what operator does the correction reach the field, and is that operator local?* A correction that reaches a cell uniformly, or through a global solve, will buy area with disconnected territory.
4. **Do not price a speedup in iterations you did not have to run.** Most of the 32× was an optimizer dying early. A result that looks like a large speedup on a hard problem should be checked for a collapsed line search before it is believed.

Related documents

- `docs/reference/winner_take_all_partition_gap.md` — the standing explanation; §9b states the locality criterion this report confirms, and §4c records the screening test.
- `docs/experiments/04-territory-aware-highn-validation/` — the other mechanism that failed the same criterion.
- `docs/math/04-phase1-timing-profile/` — the projection cost that motivated the attempt.
- Code: `src/optimization/projection.py` (`project_rows_onto_simplex`), `src/optimization/pgd_optimizer` (penalty, gate, stall guard), `src/partition/balanced_readout.py` (what made the attempt thinkable).